

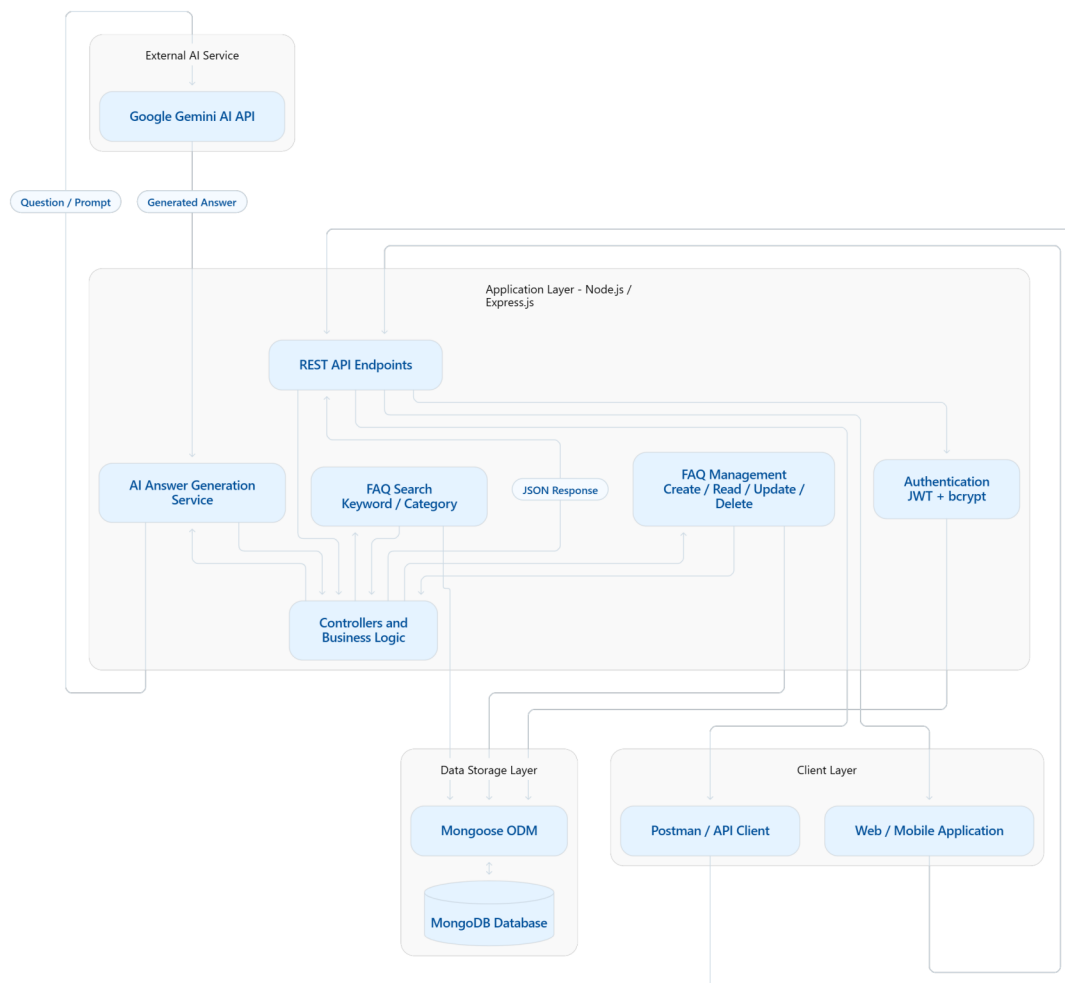
Technology Stack (Architecture & Stack)

Project Name	AI FAQ Assistant API
Team ID	SWTID-2026-3342
Date	30TH SEPTEMBER 2026

1. Technical Architecture

The AI FAQ Assistant API follows a layered architecture in which the user interface or API client communicates with a Node.js and Express.js backend. The backend handles authentication, FAQ management, search, and AI-powered answer generation. MongoDB stores user accounts, FAQs, and categories, while Google Gemini AI generates answers to user questions.

1.1 Architectural Diagram



1.2 Architecture Description

The architecture consists of the following layers:

- **Client Layer:** Web or mobile applications and API testing tools such as Postman send requests to the backend.
- **Application Layer:** Node.js and Express.js handle incoming requests, route them to controllers, and execute the business logic.
- **Authentication Layer:** JWT provides token-based authentication, while bcrypt securely hashes and verifies user passwords.
- **FAQ Management and Search:** Users can create, retrieve, update, delete, and search FAQs. Category-based filtering helps organize the information.
- **Database Layer:** MongoDB stores user details, FAQ records, and categories. Mongoose provides object data modeling between the application and database.
- **External AI Interface:** The AI service sends user questions to Google Gemini and receives generated answers.
- **Response Layer:** The API returns results, generated answers, or error messages in JSON format.

1.3 Infrastructure Demarcation

Local Development Environment

Node.js, Express.js, MongoDB (local or configured database), Mongoose, and Postman. Developers use this environment to build and test the API.

External / Cloud Services

Google Gemini AI API provides AI-generated answers. MongoDB Atlas and a cloud application host can be used for cloud deployment, if configured.

2. Table 1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	Allows users and developers to interact with the API. A web or mobile frontend can consume the API.	HTML, CSS, JavaScript / Postman
2	Application Logic – 1	Handles incoming HTTP requests, API routing, and request-response processing.	Node.js, Express.js
3	Application Logic – 2	Manages user registration, login, password verification, and token-based authentication.	JWT, bcrypt
4	Application Logic – 3	Implements FAQ creation, retrieval, updating, deletion, keyword search, and category filtering.	JavaScript, Express.js, Mongoose
5	Database	Stores user information, FAQs, and category records.	MongoDB
6	Cloud Database	Provides managed cloud storage for application data, if deployed to the cloud.	MongoDB Atlas (proposed option)
7	File Storage	The current core API does not require a separate file-storage service. Application configuration and logs may be stored locally or through a hosting service.	Local filesystem / hosting storage
8	External API – 1	Sends user questions to an external AI service and receives generated answers.	Google Gemini API
9	External API – 2	No second external API is required for the core project. Additional services can be integrated if needed.	Not applicable
10	Machine Learning Model	Generates natural-language answers based on user questions and the prompt provided by the application.	Google Gemini AI model via API
11	Infrastructure (Server / Cloud)	Runs the backend API and connects it to the database and external AI service.	Local Node.js server; cloud hosting can be added

Local Server Configuration

- Runtime: Node.js
- Backend framework: Express.js
- Database: MongoDB local instance or configured MongoDB database
- Data access: Mongoose
- API testing: Postman
- Authentication: JWT and bcrypt
- AI integration: Google Gemini API

Cloud Server Configuration

For future cloud deployment, the application can be hosted on a cloud application platform, with MongoDB Atlas used as the managed database. The Google Gemini API is accessed as an external service.

Note: Cloud hosting and MongoDB Atlas are proposed deployment options, not claims that the project is already deployed there.

3. Table 2: Application Characteristics

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	The application uses open-source technologies for backend development, API routing, and database integration.	Node.js, Express.js, Mongoose
2	Security Implementations	Passwords are hashed using bcrypt. JWT is used for authentication and protecting authorized API endpoints. Input validation and secure handling of API keys should be applied.	bcrypt, JWT, Express.js
3	Scalable Architecture	The application separates API routes, controllers, business logic, and database operations. This modular structure makes it easier to maintain and extend as the number of users and FAQs increases.	Node.js, Express.js, MongoDB, Mongoose
4	Availability	The API can be deployed on a reliable hosting platform. Database backups, monitoring, and restart policies can improve availability. Redundant servers and load balancing may be added for production deployment.	Cloud hosting, MongoDB Atlas (proposed)
5	Performance	FAQ queries should be optimized, and suitable database indexes can improve search performance. Efficient API processing and controlled calls to the AI service can reduce unnecessary delays.	MongoDB indexes, Express.js, Gemini API

4. Technology Stack Summary

Backend	Node.js + Express.js
Programming Language	JavaScript
Database	MongoDB
Database ODM	Mongoose
Authentication	JWT
Password Security	bcrypt
AI Integration	Google Gemini API
API Testing	Postman
Data Format	JSON

5. Conclusion

The AI FAQ Assistant API uses a modular, layered architecture to provide automated customer support and efficient FAQ management. Node.js and Express.js form the backend, MongoDB stores application data, and Mongoose simplifies database operations. JWT and bcrypt provide authentication and password security, while Google Gemini AI enables natural-language answer generation.

The proposed architecture supports future integration with web and mobile applications. Its modular design also allows additional features, database optimization, cloud deployment, and scaling to be introduced as project requirements grow.